

并行算法

穿过了 4 个困难而又精彩的理论专题，我们来到了这门课程的尾声。在期末各大课程大作业等压力骤增的情况下，这门课程最后两章难度的下降给了诸位温柔的怀抱。言归正传，本讲的主题在这门课程中相对比较独立：我们将进入一个全新的领域，讨论高性能计算在算法方面的基础优化方法，即设计所谓并行算法。本讲将在几个例子中讨论设计并行算法的几种基本思想与技巧。

14.1 何谓“并行”

并行在不同层面有不同的含义。在计算机体系结构中，会见到如下三种并行方式：

1. 指令级并行 (Instruction-Level Parallelism, ILP)：在一个处理器内部，多条指令的执行过程发生重叠，例如流水线让不同指令同时处于不同阶段。超标量处理器有多个执行部件，可以同时处理没有依赖关系的指令。VLIW 则由编译器把可同时执行的操作组织到一条超长指令中。
2. 数据级并行 (Data-Level Parallelism, DLP)：把相同操作同时作用于多个数据项。例如 SIMD (Single Instruction, Multiple Data) 向量加法中，各对对应元素相加互不干扰。
3. 线程级并行 (Thread-Level Parallelism, TLP)：这也是本讲将要讨论的内容，指让多个线程的工作同时向前推进。它可以利用多核、多处理器，也可以利用一个核心上的同时多线程 (SMT)。使用 SMT 时，同一个核心上的线程共享该核心的执行资源。

这里需要澄清三个相似但又不一致的词语：并发、并行和分布式。并发关注多个任务的执行在时间上交叠，可能真正同时执行，但也可能只是在单核上交替执行；并行强调同一时刻有多个操作在执行；分布式强调计算分布在通过网络通信的多个结点上。

14.2 并行算法基本模型

14.2.1 加速比与共享内存

使用并行算法显然是希望利用现代计算机多核的优势，同时执行一些可以不互相干扰的算法部分，加速算法的运行，因此有一个重要的参数称为加速比。

定义 14.1 (加速比). 加速比 (Speedup) 是指在并行计算中，对于同一个输入，使用 p 个处理器相对于使用一个处理器时的性能提升比例，即

$$S(p) = \frac{T_1}{T_p},$$

其中 T_1 是使用一个处理器时的运行时间， T_p 是使用 p 个处理器时的运行时间。

如果工作没有增加且能平均分给 p 个处理器，最理想的加速比就是 p ，也就是运行时间缩短到原来的 $1/p$ 。考虑一个很简单的例子，两个 n 阶方阵 A 与 B 相乘得到矩阵 C 。普通算法需要 n^3 次乘法和 $n^2(n-1)$ 次加法。如果有 n^2 个处理器，那么有一个自然的并行策略，也就是让每个处理器负责结果矩阵 C 的一个元素的计算，而每个元素的计算互不干扰。只计加法和乘法时，每个处理器需要完成 $2n-1$ 次运算，因此加速比就是 n^2 ，看起来非常完美。

然而实际情况中，使用多个处理器意味着额外的通信开销。如果处理器并未分配到相同的工作量，先做完的处理器还要等待，就会造成负载不均衡 (load imbalance)，再次降低实际速度；此外，部分代码依赖原有顺序执行，也不能完全并行。接下来的模型会暂时忽略通信等开销，让我们先讨论算法本身能够做到多快。

事实上，上述讨论还隐含了一个条件，即多个处理器可以同时读取同一个内存位置。比如同时得到 C 的第一行的所有元素需要同时访问 A 的第一行的所有元素。这种共享内存的计算方式需要通过模型明确规定，下面介绍本讲使用的 PRAM (Parallel Random Access Machine) 模型，它是 RAM 模型在共享内存系统上的扩展，规定了如下三种内存共享方式：

1. EREW (Exclusive Read, Exclusive Write)：每一步中，同一个位置至多由一个处理器读取，也至多由一个处理器写入。
2. CREW (Concurrent Read, Exclusive Write)：允许多个处理器同时读取同一个位置，但仍只允许一个处理器写入。
3. CRCW (Concurrent Read, Concurrent Write)：同时读取和同时写入都允许。由于不同处理器可能写入不同的值，还需要约定如何处理写入冲突：
 - Common：只有各处理器写入相同的值时，才允许它们同时写入。
 - Arbitrary：写入值可以不同，任意选取其中一个作为结果，算法必须在每一种选择下都能正确运行。
 - Priority：预先给处理器规定优先级，保留优先级最高者写入的值。

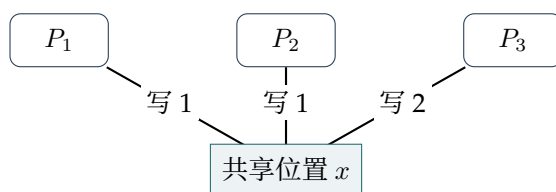


图 14.1 三个处理器同时向 x 写入 1, 1, 2。这不符合 Common 的要求；Arbitrary 可以保留 1 或 2；若编号小的处理器优先，Priority 会保留 1。

PRAM 忽略了缓存、内存带宽和远距离通信，因此模型中得到的时间不一定是实际机器上的运行时间。但我们的目的是先讨论并行算法的基本思想，用这个简化模型就能看清哪些操作可以同时进行，以及算法还受到哪些依赖关系的限制。

14.2.2 加速比的三种模型

前面的矩阵乘法可以将工作平均分给各个处理器，但一般程序中往往有一些操作必须串行执行，这会怎样影响加速比呢？本节将讨论三种针对这一问题的模型。直观而言，如果原来总时间的 5% 必

须串行执行，即使其余部分瞬间完成，总时间也至少是原来的 5%，加速比最多为 20。这种现象称为阿姆达尔定律（Amdahl's Law）。注意，以下讨论的加速比都是程序针对同一个输入的加速比。

定理 14.2 (阿姆达尔定律). 设程序在一个处理器上运行需要 $t > 0$ 时间，其中必须串行执行的部分占比为 $f \in [0, 1]$ ，那么使用 p 个处理器的最大加速比 $S(p)$ 满足

$$S(p) = \frac{1}{f + (1 - f)/p}.$$

证明. 由于必须串行执行的部分需要 ft ，其余部分需要 $(1 - f)t$ 。使用 p 个处理器时，前一部分时间不变，后一部分缩短为 $(1 - f)t/p$ ，因此

$$S(p) = \frac{t}{ft + (1 - f)t/p} = \frac{1}{f + (1 - f)/p}.$$

□

当 $p \rightarrow \infty$ 时， $S(p) \rightarrow 1/f$ ，看起来这是一个很糟糕的结论，因为如果一个程序可并行的部分占到 90%，看起来非常多，但加速比最多只能达到 10 倍，是一个并不大的数字，因此一定程度上给研究并行计算的人们带来了挫败感，因为这个定律让人们认为并行可能不是一个很有前途的方向。

然而我们可以怀疑这一定律的合理性：阿姆达尔定律认为只增加处理器数量并不会对并行加速结果有明显的提升，其隐含假设是，越来越多的处理器上执行同一个固定计算。然而在实际中情况并非如此，人们根据可用处理器的数量来调整问题规模的大小，处理器很多的时候可以处理更多的数据。并且有一个现实的假设是，串行部分可以是独立于问题大小的，因此数据量越大，并行部分越大，而阿姆达尔定律则是假设数据集的大小是固定的。首个打破阿姆达尔定律这一假设的是 John L. Gustafson，他在 1988 年于 Communications of the ACM 上发表论文提出了如下定理：

定理 14.3 (古斯塔夫森定律 (Gustafson's Law)). 设某个程序使用 p 个处理器并行执行时，串行部分使用的时间为 f_1 ，并行部分使用的时间为 f_2 ，那么使用 p 个处理器的最大加速比 $S(p)$ 满足

$$S(p) = \frac{f_1 + f_2 \cdot p}{f_1 + f_2}.$$

证明. 由假设可知，使用 p 个处理器并行执行的时间为 $T_p = f_1 + f_2$ ，使用一个处理器串行执行的时间最大为 $T_1 = f_1 + f_2 \cdot p$ ，因此加速比为

$$S(p) = \frac{T_1}{T_p} = \frac{f_1 + f_2 \cdot p}{f_1 + f_2}.$$

□

这里的关键就是允许问题规模扩大，因此没有规定串行和并行时间所占的比例，而是可以根据处理器的数量调整它们各自的占比。当 $f_2 \rightarrow \infty$ 时， $S(p) \rightarrow p$ ，因此增加处理器仍然可以很有价值。

阿姆达尔定律和古斯塔夫森定律看起来似乎水火不容，但 1990 年，密西根州立大学的博士生孙贤和（现为伊利诺理工大学教授）与倪明选（现为香港科技大学教授）在 Supercomputing 会议上提出了一种并行加速模型，成功地统一并扩展了 Amdahl 定律与 Gustafson 定律。

定理 14.4 (孙-倪定律 (Sun-Ni's Law)). 设某个程序串行部分占比为 f , 并行部分占比为 $1 - f$, 假设扩大问题后, 串行部分保持不变, 可并行部分增加到原来的 $G(p)$ 倍, 那么使用 p 个处理器的最大加速比 $S(p)$ 满足

$$S(p) = \frac{f + (1 - f) \cdot G(p)}{f + \frac{(1 - f) \cdot G(p)}{p}}.$$

证明. 扩大后的问题若用一个处理器完成, 需要 $f + (1 - f)G(p)$ 时间; 改用 p 个处理器, 只能把可并行的那一部分除以 p , 因此需要 $f + (1 - f)G(p)/p$ 时间。两者相除即得上式。□

事实上, $G(p)$ 的定义与内存有关: 当增加处理器时, 内存也随之增加, 因此我们可以利用这些内存处理更多数据。假设可用内存为 M 时, 可并行部分的工作量是 $g(M)$, 增加处理器后内存变为 pM , 则工作量变为原来的 $G(p) = g(pM)/g(M)$ 倍。例如 $g(M) = cM^r$ 时, 就有 $G(p) = p^r$ 。当 $r = 1$ 时, 内存和工作量按相同倍数增长, 即 $G(p) = p$, 这就是古斯塔夫森定律的情况; 当 $r = 0$ 时, 工作量不随内存增加而增长, 即 $G(p) = 1$, 这就是阿姆达尔定律的情况。这样, 三种模型就联系起来了。

14.2.3 工作量与深度

前面几个模型都把程序分成串行和并行两部分, 但很多程序并不能这样干净地分开。例如求和时, 某个加法只需等待自己的两个输入算完, 其他无关的加法即使还没完成, 也不影响它继续计算。因此, 我们需要逐项考虑操作之间的依赖关系。

把每个单位时间操作画成一个结点, 如果操作 v 要使用操作 u 的结果, 就从 u 向 v 连一条有向边, 由此得到一张有向无环图。沿着图中的一条路径, 各个操作必须依次完成, 其中最长的依赖链称为关键路径 (Critical Path)。即使有无穷多个处理器, 也只能等前一个操作完成后再执行下一个, 所以关键路径自然限制了算法能够达到的速度。

定义 14.5 (工作量与深度). 并行算法的工作量 (work) W 是完成一次计算所需的全部单位操作数, 深度 (depth, 也称 span) D 是关键路径上包含的操作数。因此, 忽略调度和通信开销时,

$$W = T_1, \quad D = T_\infty.$$

对于非空计算, W/D 称为算法的平均并行度 (average parallelism)。

现在我们知道了 T_1 和 T_∞ , 但实际只有 p 个处理器时需要多久呢? 下面的布伦特定理将一般的 T_p 与工作量、深度联系起来。

定理 14.6 (布伦特定理 (Brent's Theorem)). 设一个并行计算问题的工作量为 W , 关键路径长度为 D , 那么使用 p 个处理器的并行时间具有如下上下界:

$$\frac{W}{p} \leq T_p \leq \frac{W - D}{p} + D.$$

证明. 下界是显然的, 因为 $T_p \geq T_\infty$, 而 T_∞ 也是受到理想加速比的限制的, 因此 $T_p \geq W/p$ 。

对于上界，因为关键路径长度为 D ，因此整个算法的执行可以分为 D 个内部并行，互相之间串行的阶段，设每个阶段的工作量为 W_i ，则有 $W = \sum_{i=1}^D W_i$ 。使用 p 个处理器时，每一个阶段所需的时间为 $\lceil W_i/p \rceil$ ，向上取整的原因在于，每个阶段的工作量可能不能被 p 整除，比如大任务被分成了 7 个子任务，7 个任务分配给 3 个处理器，那么每个处理器分配到的任务数应当是 2, 2, 3。因此需要的总时间就是

$$T_p = \sum_{i=1}^D \left\lceil \frac{W_i}{p} \right\rceil = \sum_{i=1}^D \left(\left\lfloor \frac{W_i-1}{p} \right\rfloor + 1 \right) \leq \sum_{i=1}^D \left(\frac{W_i-1}{p} + 1 \right) = \frac{W-D}{p} + D.$$

□

14.2.4 通信速度带来的限制

前面一直在讨论增加处理器能否使算法更快，但很多处理器是否真的能像 PRAM 中那样协作，也值得考虑。处理器之间需要通信，信息传播速度又有上界，因此离得太远的处理器即使完成了计算，也可能来不及将结果传到需要它的位置。下面通过一个简化的估计，说明这种物理限制如何影响运行时间。

假设处理器分布在三维空间中，每单位体积内的处理器数有固定上界，每个处理器每单位时间只能完成常数操作，信息传播速度也有固定上界。所有计算都从时刻 0 开始，并要求在时刻 T 将最终结果汇集到某个指定位置。对于时刻 t 完成的操作，留给它传递结果的时间只有 $T-t$ ，所以只有距离输出位置 $O(T-t)$ 以内的处理器，才来得及对最终结果产生影响。这些处理器位于一个半径为 $O(T-t)$ 的球中，其数量至多为 $O((T-t)^3)$ ，单位时间内能够完成的操作也至多为这么多。对整个计算过程求和，可知在时间 T 内，能够影响最终结果的操作数至多为

$$O\left(\int_0^T (T-t)^3 dt\right) = O(T^4).$$

如果已经知道最终结果至少需要 Q 次本次计算中的操作共同决定，根据 $Q = O(T^4)$ ，有 $T = \Omega(Q^{1/4})$ 。若串行算法需要 $\Theta(Q)$ 时间，相应的加速比至多为 $O(Q^{3/4})$ 。它说明，即使算法中有大量工作可以并行，实际通信也会限制这些处理器能否及时参与计算。

14.3 前缀和问题

前面的讨论介绍了并行计算的基本模型，接下来将在这些模型上讨论具体的算法设计。通常有五种设计并行算法的范式，包括平衡二叉树、划分范式、双对数范式、加速级联范式以及流水线范式，其中流水线范式不展开介绍，感兴趣的读者可以阅读参考资料中 Uzi Vishkin 的 note。我们先从基本的求和问题开始，再利用求和过程中得到的结果，进一步计算数组的全部前缀和，体会到并行算法设计的基本思想。

14.3.1 先从下往上求和

给定数组 $A[1 \dots n]$ ，我们希望让多个处理器并行求出所有元素的和。最自然的办法是把相邻两个元素分为一组，各组同时相加，然后将相邻两组的和再相加，如此继续，直到只剩下整个数组的和。把这个过程画出来，恰好是一棵从叶子向根逐层计算的平衡二叉树，如图 14.2 所示。为了让每一层都能两两分组，当 $n > 0$ 时，要将数组长度补到 $N = 2^{\lceil \log_2 n \rceil}$ ，新增的位置全部填 0。这样不会改变总和，且 $n \leq N < 2n$ ，元素数最多增加一倍。

将叶子所在层记为第 0 层，向上一层编号就增加 1，根所在层为 $H = \log_2 N$ 。同一层的结点从左到右编号， $B(h, i)$ 保存第 h 层第 i 个结点下面所有叶子的和，也就是

$$B(h, i) = \sum_{k=(i-1)2^h+1}^{i2^h} A[k].$$

另一方面， $B(h, i)$ 也可以由它的两个孩子结点的和得到：

$$B(h, i) = B(h-1, 2i-1) + B(h-1, 2i).$$

按层从下往上计算时，同一层的结点只使用上一层已经求出的结果，所以可以并行完成。

下面用伪代码描述这一过程，其中 **pardo** 表示对列出的各个下标并行执行后面的操作，并在这些操作全部完成后继续执行下一条语句。因此，同一层的各个结点可以同时求和，但必须等这一层算完，才能开始计算上一层。

算法 14.1 SumTree

输入： $n \geq 1$ ，数组 $A[1 \dots n]$ ；字长足以存储和

输出： 返回 B 、 N 、 H ，总和为 $B(H, 1)$

```

1:  $N \leftarrow 2^{\lceil \log_2 n \rceil}$ ;  $H \leftarrow \log_2 N$ 
2: pardo  $i = 1, \dots, N$ :  $B(0, i) \leftarrow A[i]$  若  $i \leq n$ ，否则置 0
3: for  $h = 1, \dots, H$  do
4:   pardo  $i = 1, \dots, N/2^h$ :  $B(h, i) \leftarrow B(h-1, 2i-1) + B(h-1, 2i)$ 
5: end for
6: return ( $B, N, H$ )

```

最后 $B(H, 1)$ 就是全部元素之和。每一层的不同结点读取的是不同的孩子，写入的位置也不同，所以使用 EREW 模型即可。树中共有 $2N - 1$ 个结点，每个结点只做常数工作，因此总工作量为 $O(N) = O(n)$ 。同一层可以同时计算，只需依次完成 $H + 1$ 层，所以深度为 $O(\log n)$ 。

14.3.2 再从上往下计算前缀

下面要讨论的是前缀和问题，即给定一个长度为 n 的数组 A ，需要设计并行算法，求出 A 的前 k 个元素的和，其中 k 取遍 1 到 n 。如果对每个前缀分别调用刚才的算法，虽然都能得到答案，但同一批元素会被反复相加，造成很多重复工作。我们希望继续利用已经求出的 B ，一次得到全部前缀和。

关键在于给每个树结点再保存一个值 $C(h, i)$ ，表示从数组开头一直加到这个结点下面最右边的叶子，即

$$C(h, i) = \sum_{k=1}^{i2^h} A[k].$$

这样，最后一层的 $C(0, i)$ 就是从第 1 项到第 i 项的前缀和。根结点的 C 显然等于整个数组的和，而这个结果已经保存在 $B(H, 1)$ 中，接下来只需考虑怎样从根向下计算其他结点的 C 。

结合图 14.2，可以得到如下三个观察：

1. 如果 $i = 1$ ，这个结点下面的叶子本来就从数组第一项开始，因此 $B(h, 1)$ 和 $C(h, 1)$ 加的是同一段元素，直接令 $C(h, 1) = B(h, 1)$ 即可。

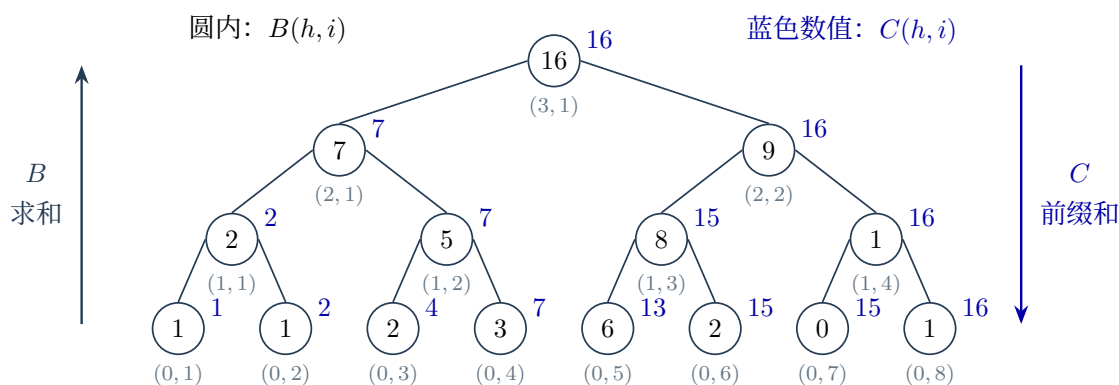


图 14.2 数组 (1, 1, 2, 3, 6, 2, 0, 1) 的求和与前缀和。圆内为自底向上求出的 $B(h, i)$ ，蓝色数值为自顶向下求出的 $C(h, i)$ ，结点下方标出 (h, i) 。最下层的蓝色数值就是全部前缀和。

2. 如果 i 是偶数，这个结点就是其父结点的右孩子。它下面最右边的叶子，也就是父结点下面最右边的叶子，所以二者的 C 相同，即 $C(h, i) = C(h + 1, i/2)$ 。
3. 如果 $i > 1$ 且为奇数，这个结点就是其父结点的左孩子。它的 $B(h, i)$ 只包含自己下面的叶子，还缺少数组开头到这组叶子之前的部分。这部分恰好是上一层第 $(i - 1)/2$ 个结点的前缀，因此

$$C(h, i) = C(h + 1, (i - 1)/2) + B(h, i).$$

例如 $C(1, 3)$ 要求前六项的和, $B(1, 3) = 6 + 2 = 8$ 只包含第五、六项, 再加上 $C(2, 1) = 1 + 1 + 2 + 3 = 7$, 便得到 $C(1, 3) = 15$ 。

下面的代码将偶数位置和奇数位置分开处理，这是为了满足 EREW 的要求。例如 $C(h, 2)$ 和 $C(h, 3)$ 都要读取 $C(h + 1, 1)$ ，若同时执行就出现了并发读取。

算法 14.2 PrefixSum

输入：数组 $A[1 \dots n]$, $n \geq 0$

输出：返回包含当前项的全部前缀和

```

1: if  $n = 0$  then
2:   return 空数组
3: end if
4:  $(B, N, H) \leftarrow \text{SUMTREE}(A)$ ;  $C(H, 1) \leftarrow B(H, 1)$ 
5: for  $h = H - 1, H - 2, \dots, 0$  do
6:   pardo 本层所有偶数  $i$ :  $C(h, i) \leftarrow C(h + 1, i/2)$ 
7:    $C(h, 1) \leftarrow B(h, 1)$ 
8:   pardo 本层所有奇数  $i > 1$ :  $C(h, i) \leftarrow C(h + 1, (i - 1)/2) + B(h, i)$ 
9: end for
10: pardo  $i = 1, \dots, n$ :  $S[i] \leftarrow C(0, i)$ 
11: return  $S$ 

```

算法在树上先从下往上，再从上往下各计算一遍，每个结点都只做常数工作，因此总工作量仍为 $O(n)$ ，深度也是 $O(\log n)$ 。

事实上，这一方法适用于任意有单位元的结合运算，例如固定阶矩阵乘法，只需保持元素的先后

顺序，将加法换成乘法、补零换成补单位矩阵即可。前缀和还可以用来按原顺序筛选数组元素，例如用 1 标记要保留的项、0 标记要删除的项，对标记求前缀和后，每个保留项对应的前缀和就是它在新数组中的位置。

14.4 归并问题

接下来将介绍第二种并行算法的设计思路。我们要讨论的问题与归并排序中的“归并”步骤相同，即将两个已经排好序的数组 $A[1 \dots n]$ 、 $B[1 \dots m]$ 合并成一个有序数组 $C[1 \dots n + m]$ 。两个数组可以不等长，也允许出现相同的值。

14.4.1 简单的想法

既然要设计并行算法，那么如果能同时找到所有元素在最终数组 C 中的位置，再同时将它们放到正确的位置上，就可以很快完成归并。这里的关键是找到每个元素前面一共有多少个元素。例如 $A[i]$ 前面已经有 $i - 1$ 个来自 A 的元素，只要再数出有多少个来自 B ，就能确定它在 C 中的位置。

对于相同的值，我们约定 A 中的元素放在 B 中的元素之前，同一数组内部则保持原有顺序。于是，对 $A[i]$ 只需数出 B 中严格小于它的元素，而对 $B[j]$ 则需要数出 A 中小于或等于它的元素。分别记为 $R_A[i] = |\{j : B[j] < A[i]\}|$ ， $R_B[j] = |\{i : A[i] \leq B[j]\}|$ 。因此， $A[i]$ 前面共有 $i - 1 + R_A[i]$ 个元素，最终位置就是 $i + R_A[i]$ ；同理， $B[j]$ 的最终位置是 $j + R_B[j]$ ，即

$$C[i + R_A[i]] = A[i], \quad C[j + R_B[j]] = B[j].$$

由此，归并就转化为在另一数组中查找位置的问题，而这一步可以用二分查找完成。记两个数组的总长度为 $L = n + m$ 。所有二分查找可以同时进行，每次需要 $O(\log L)$ 时间，所以深度为 $O(\log L)$ 。共有 L 个元素分别查找，总工作量为 $O(L \log L)$ 。不同查找可能同时读取同一个中点，因此使用 CREW 模型。

当然也可以不使用并行算法，进行普通的顺序归并，让两个指针分别从数组开头向后走，每次取较小的元素，直到一个数组用完后，再输出另一个数组剩下的部分，工作量和深度都是 $O(L)$ 。这两种方法都不令人满意，因为前者的总工作量太大，而后的深度太大。我们希望找到一种折中的办法，让它们取长补短，这就引出了划分范式。

14.4.2 使用划分范式

划分范式 (Partitioning Paradigm) 的想法是，先将原问题划分为许多个较小的子问题，再同时处理这些子问题。对于归并，我们可以只对少量元素使用二分查找，基于此把输出分成若干段，然后每一段内部用普通的顺序归并，而不同段之间同时执行。这样既减少了二分查找的次数，又避免了从头到尾依次归并。

具体地，在两个数组中每隔 b 项取一个元素 (b 的具体取值等分析复杂度时再确定)，也就是选取下标 $1, 1 + b, 1 + 2b, \dots$ 处的元素，称为分界元素。先用二分查找求出各个分界元素在另一数组中有多少项排在它前面，从而确定它在结果中的位置。再给每个分界元素分配一个任务，负责输出从它开始到下一个分界元素之前的所有元素。

以图 14.3 中的 $A[1] = 4$ 为例，二分查找发现 B 中有三项排在它之前，所以这个任务从 $A[1]$ 和 $B[4]$ 开始比较，通过普通归并依次输出 4、5、6。接下来本应输出 $B[5] = 7$ ，但它也是分界元素，有

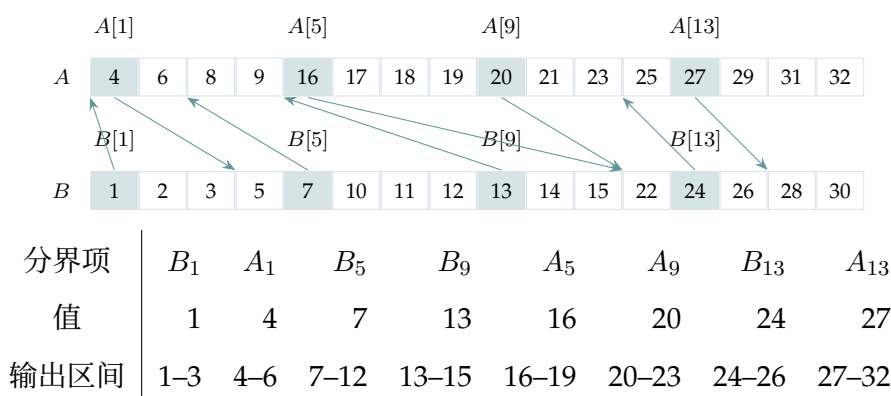


图 14.3 每隔 $b = 4$ 项选一个分界元素，并在另一数组中二分查找其位置。下表按合并后的顺序列出分界元素，以及各自负责输出的区间。

自己的任务负责，因此当前任务就停在这里。同理，从 $B[9] = 13$ 开始的任务会输出 13、14、15，遇到分界元素 $A[5] = 16$ 时停止。

这样，每个任务只需知道自己的起点，然后按普通归并运行，直到遇到下一个分界元素就停止。判断是否遇到分界也很简单，检查当前下标是否为 $1, 1 + b, 1 + 2b, \dots$ 即可。

下面将查找起点和局部归并写在同一个算法中。每个任务使用独立的指针 i, j ，按前面约定的顺序归并，相同值优先取 A 中的元素。输出 $A[i]$ 时记下 $R_A[i] = j - 1$ ，输出 $B[j]$ 时记下 $R_B[j] = i - 1$ ，就能同时得到排名数组。

算法 14.3 PartitionMerge: 划分后并行归并

输入： 非降数组 $A[1 \dots n]$, $B[1 \dots m]$

输出： 归并结果 C

- 1: $L \leftarrow n + m$
- 2: **if** $L = 0$ **then**
- 3: **return** 空数组
- 4: **end if**
- 5: $b \leftarrow \max\{1, \lceil \log_2 L \rceil\}$
- 6: 在两数组中选取下标为 $1, 1 + b, 1 + 2b, \dots$ 的分界元素
- 7: **pardo** 对每个分界元素执行以下步骤:
- 8: 若为 $A[s]$, 令 $i \leftarrow s$, 二分查找 B 中小于 $A[s]$ 的项数, 令 j 为该项数加 1
- 9: 若为 $B[t]$, 令 $j \leftarrow t$, 二分查找 A 中小于或等于 $B[t]$ 的项数, 令 i 为该项数加 1
- 10: 从 $A[i]$ 、 $B[j]$ 开始顺序归并, 依次写入 $C[i + j - 1]$ 起的各位置
- 11: 在输出下一个分界元素之前停止; 若没有下一个分界元素, 则归并至结束
- 12: **return** C

考虑上述算法的复杂度。一个数组每隔 b 项就有一个分界元素，所以当前任务最多从这个数组取出 b 项，再向后就一定会遇到下一个分界元素而停止。两个数组合计至多取出 $2b$ 项，因此一次局部归并只需 $O(b)$ 时间。分界元素共有 $q = \lceil n/b \rceil + \lceil m/b \rceil = O(L/b)$ 个。第一步对它们分别二分查找，深度为 $O(\log L)$ ，工作量为 $O(L/b \cdot \log L)$ ；第二步同时归并各段，深度为 $O(b)$ ，而每个元素只输出一次，所以合计工作量为 $O(L)$ 。

现在就可以确定 b 了。为了让第二步深度不超过对数级，应取 $b = O(\log L)$ ；为了让第一步工作量不超过线性级，又希望 b 至少与 $\log L$ 同阶。因此，代码取 $b = \max\{1, \lceil \log_2 L \rceil\}$ ，便同时得到

$$W = O(L), \quad D = O(\log L).$$

二分查找和相邻任务的停止检查可能同时读取同一个元素，但每个输出位置只由一个任务写入，所以采用 CREW 模型即可。总而言之，这一算法结合了前面两种方法的优点：在长数组上只对少量元素使用二分查找，再在分出的短数组内顺序归并。选好每段长度后，二分查找不会带来过多工作，顺序归并也不会拖长并行时间。

14.5 寻找最大元素

下面希望设计并行算法，找到一个数组中的最大元素。我们将讨论双对数范式，它是二叉树方法的一种扩展，然后讨论加速级联范式，将不同算法的优点结合起来，最后再尝试用随机算法进一步缩短并行时间。

14.5.1 简单的想法

在讨论复杂的范式前，先来讨论一些简单的想法。第一种最自然的办法就是类似于前面求和问题的做法，构造一棵二叉树，初始元素两两分组比较，然后逐层上递，最终得到最大值。只需将 SumTree 中的加法换为取最大值，就能得到深度 $O(\log n)$ 、总工作量 $O(n)$ 的 EREW 算法。

然而我们可以更激进：为什么不一次性把所有元素之间都比较一次呢？如果不考虑总工作量会很大的问题，这种方式是可行的。先给每个元素准备一个初始为 0 的标记，再并行比较每一对元素，只要某个元素较小，就将它的标记改为 1。这样，最大元素从未输过，标记仍为 0，而其他元素至少会输给最大元素一次，最后只需并行判断每个元素的标记，输出标记为 0 的元素即可。这里有一个需要注意的细节。在并行比较每一对元素时，可能有多个处理器同时修改同一个较小元素的标记，但它们都只会写入 1，因此使用 Common CRCW 就能完成。

不难理解，这一算法的深度为 $O(1)$ ，但总工作量为 $\Theta(n^2)$ ，因为有 $n(n-1)/2$ 对元素需要比较。这与二叉树方法又形成了两个极端：一个工作量小但深度较大，另一个深度小但工作量较大，接下来尝试让它们取长补短。

14.5.2 双对数范式

双对数范式 (Doubly-logarithmic Paradigm) 可以看作二叉树方法的一种扩展。在二叉树中，每个结点只有两个孩子，所以 n 个叶子需要 $O(\log n)$ 层。现在希望减少层数，就可以让一个结点拥有更多孩子，也就是一次分出更多子问题。

一个合适的选择是，将长度为 n 的数组分成 $\sqrt{n}$ 组，每组 $\sqrt{n}$ 项。对每一组，继续按同样的方法递归分组。图 14.4 通过树形象地给出了这种分组的过程。递归的 base case 是当组内只剩不多于两个元素，可以直接比较。各组的计算互不依赖，因此可以并行进行。递归返回的时候，使用此前介绍的两两比较方法，深度只需 $O(1)$ ，工作量为 $O((\sqrt{m})^2) = O(m)$ ，其中 m 是当前每组的长度。

讨论复杂度，先考虑 $n = 2^{2^h}$ 的情形，这样递归中的平方根都是整数。由上述步骤可以直接写出

$$\begin{aligned} D(n) &= D(\sqrt{n}) + O(1), \\ W(n) &= \sqrt{n} W(\sqrt{n}) + O(n). \end{aligned}$$

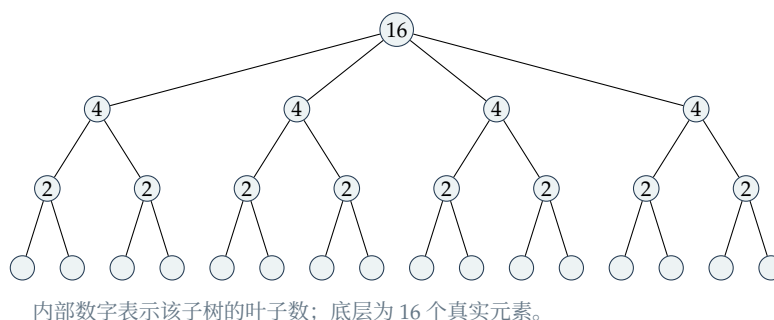


图 14.4 含 16 个元素的双对数树。根将它们分成四组，每组四项；各组继续分成两组，每组两项，最下层的叶子表示原数组元素。

对于深度,可以沿着递归树向下看。每向下一层,元素数就开一次平方根,因此第 s 层每个子问题有 $n^{1/2^s}$ 项。当这个数量降到 2 时,有 $2^s = \log_2 n$, 也就是 $s = \log_2 \log_2 n$, 所以深度 $D(n) = O(\log \log n)$ 。对于工作量,同样可以使用递归树。根处要比较 $\sqrt{n}$ 个最大值,共需 $O(n)$ 工作;下一层有 $\sqrt{n}$ 个子问题,每个合并答案需要 $O(\sqrt{n})$ 工作,合计仍为 $O(n)$ 。继续向下,每层总工作都是 $O(n)$ 。一共有 $O(\log \log n)$ 层,因此 $W(n) = O(n \log \log n)$ 。

实际输入不一定能这样整齐地开平方根。可以每次取组长 $b = \lceil \sqrt{m} \rceil$, 最后一组不足 b 项时按实际长度处理即可。这里不需要补成某个 2^{2^h} , 如果直接补齐,那么长度刚超过 256 时就要补到 65536, 新增的元素可能使工作量增加太多。还需要说明取整不会增加太多层。当 $m \geq 16$ 时,有 $\lceil \sqrt{m} \rceil \leq m^{3/4}$, 因此经过 $O(\log \log n)$ 层,每组就不足 16 项,再递归常数层即可结束,因此一般长度也有相同的渐近界。

14.5.3 加速级联范式

我们仍然希望改进上面的算法。双对数方法已经很快,但总工作量比线性级多了一个 $\log \log n$ 因子。如果先用另一个工作量较小的方法减少元素数,再交给它处理,就有可能把这个因子抵消掉。这就是加速级联范式 (Accelerating Cascades Paradigm) 的思路,即组合不同的策略加速计算。

具体做法也很自然,先将数组分成约 $n / \log \log n$ 组,每组只有约 $\log \log n$ 项。各组内部直接顺序扫描求最大值,不同组同时执行,这一步深度为 $O(\log \log n)$,总工作量为 $O(n)$ 。然后将剩下的各组最大值交给上面介绍的双对数范式求解,此时输入只剩约 $n / \log \log n$ 项,因此可以得到

$$D(n) = O(\log \log n) + O(\log \log(n / \log \log n)) = O(\log \log n),$$

$$W(n) = O(n) + O((n / \log \log n) \cdot \log \log(n / \log \log n)) = O(n).$$

第一步顺序扫描可以在 EREW 上完成,第二步使用 Common CRCW (因为其中使用了两两比较的方法),所以整个算法使用 Common CRCW 即可。由于寻找最大值至少要让其他 $n - 1$ 个元素各输一次比较,故总工作量至少为 $\Omega(n)$,这个算法已经达到了比较模型下的最优工作量。

14.5.4 随机算法：先留下少量有希望的元素

看起来还有改进的空间,毕竟当初暴力的两两比较方法能实现 $O(1)$ 的深度。接下来将介绍一种随机算法,它可以保证以很高的概率在 $O(1)$ 的深度和 $O(n)$ 的工作量内找到最大值。

我们首先给出算法的描述,然后进行分析:

1. 第一步：从长度为 n 的数组 A 中，独立、有放回地均匀抽取 $n^{7/8}$ 项，得到新的数组 B 。这一步让 $n^{7/8}$ 个处理器各自负责抽取一项，放到 B 中各自的位置，深度为 $O(1)$ ，总工作量为 $O(n^{7/8})$ 。
2. 第二步：求出 B 中的最大值，使用确定性算法，通过如下三个子步骤实现。
 - (a) 把 B 分成 $n^{3/4}$ 个长度为 $n^{1/8}$ 的子数组，然后使用两两比较的方法，并行找到每个子数组的最大值。这一步深度为 $O(1)$ ，总工作量为 $O(n^{3/4} \cdot (n^{1/8})^2) = O(n)$ 。将这 $n^{3/4}$ 个最大值放在新数组 C 中。
 - (b) 把 C 分成 $n^{1/2}$ 个长度为 $n^{1/4}$ 的子数组，再并行找到每个子数组的最大值。这一步仍然使用两两比较，深度为 $O(1)$ ，总工作量为 $O(n^{1/2} \cdot (n^{1/4})^2) = O(n)$ 。然后将这 $n^{1/2}$ 个最大值放在新数组 D 中。
 - (c) 数组 D 直接使用两两比较的方法找到最大值。这一步深度为 $O(1)$ ，总工作量为 $O((n^{1/2})^2) = O(n)$ 。

整个第二步相当于三轮淘汰赛，每轮都留下各组的最大值，最后得到的就是所抽取的 $n^{7/8}$ 项中的最大值，记为 M 。三轮合计的深度仍为 $O(1)$ ，总工作量为 $O(n)$ 。当然最初抽取的项数也可以改为 $n^{15/16}$ ，不过这时需要四轮淘汰，每轮仍只需 $O(n)$ 的工作量。

3. 第三步：目前只得到了样本中的最大值 M ，如何进一步找到整个数组的最大值呢？我们只需继续考虑大于 M 的元素。先将长度为 $n^{7/8}$ 的数组 B 全部填入 M ，然后让 n 个处理器各自检查 A 中的一项。若这一项不大于 M ，就不必再处理；若大于 M ，就独立、均匀地选取 B 中的一个位置，将它写进去。

为什么要随机选位置呢？每个处理器只知道自己的元素是否大于 M ，还不知道应该将它放在第几个位置。为了在常数深度内完成写入，这里让它直接随机选择。多个处理器可能选中同一个位置，此时按 Arbitrary 规则保留其中一个写入值。写完之后，再用第二步的三轮比较求出更新后的 B 的最大值，作为本次算法的结果。因此，第三步也只有 $O(1)$ 的深度和 $O(n)$ 的工作量。

第三步先把 B 填成 M ，是为了保留已经找到的结果。如果 M 本来就是整个数组的最大值，就没有任何元素需要写入，最后仍会返回 M 。如果还有更大的元素，才用它们替换相应位置。

那么这样成功的概率如何呢？如果第二步得到的 M 在原数组中已经排名靠前，例如位列最大的 $n^{1/4}$ 项之中，那么大于 M 的元素就很少，第三步随机写入时发生冲突的概率也会很小。只要没有冲突，所有大于 M 的元素就都能保留下来，其中当然包括真正的最大值。即使发生冲突，也未必失败，只有真正的最大值没有留下来时才会出错。图 14.5 给出了一个失败的例子。

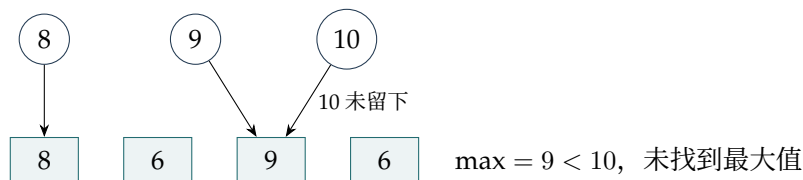


图 14.5 当前 $M = 6$ ，大于它的元素为 8、9、10。若 9 和 10 写入同一个位置，可能只留下 9，于是这一次返回 9，未能找到真正的最大值 10。

实际的 n 不一定使这些次幂都是整数。此时取 $t = \lceil n^{1/8} \rceil$ ，抽取 $s = t^7$ 项，三轮依次将每 t 项、每 t^2 项、最后剩下的 t^4 项放在一起比较即可。每轮工作量都是 $O(t^8)$ ，由 $t \leq 2n^{1/8}$ 可知 $t^8 \leq 256n$ ，所以复杂度不变。

定理 14.7 (随机最大值算法的成功概率). 上述三步算法的深度为 $O(1)$, 工作量为 $O(n)$, 返回最大值的概率至少为

$$1 - e^{-n^{1/8}} - n^{-3/8} = 1 - O(n^{-3/8}).$$

证明. 算法的复杂度已经逐步分析过, 下面只计算成功概率. 令 $q = \lceil n^{1/4} \rceil$. 只要下列两件事同时发生, 第三步就能找到最大值:

1. 第二步得到的 M 是原数组中最大的 q 项之一;
2. 所有大于 M 的元素在第三步写入时都没有冲突。

因此, 成功概率至少等于第一件事的概率, 乘以在第一件事成立的条件下第二件事的概率。

首先来看第一部分。一次抽样没有抽中最大 q 项的概率是 $1 - q/n$ 。各次抽样相互独立, 所以 s 次都没有抽中的概率为

$$\left(1 - \frac{q}{n}\right)^s \leq e^{-sq/n} \leq e^{-n^{1/8}},$$

其中用到了 $1 - x \leq e^{-x}$, $q \geq n^{1/4}$ 和 $s \geq n^{7/8}$ 。也就是说, 第一件事的概率至少为 $1 - e^{-n^{1/8}}$ 。

接下来计算第二部分。固定一份满足第一条的样本, 假设原数组中还有 r 项大于 M , 则 $r \leq q - 1 < n^{1/4}$ 。若 $r = 0$, 说明已经找到最大值, 第三步一定成功。下面考虑 $r \geq 1$ 。虽然这些元素同时写入, 但计算概率时可以依次考察它们选择的位置。首先第一个元素可以选择任意位置, 然后如果前 i 个元素已经选中了不同的位置, 那么第 $i + 1$ 个元素只需避开这 i 个位置, 就不会与它们冲突, 其概率为 $1 - i/s$ 。因此, 所有写入都没有冲突的概率为

$$\prod_{i=0}^{r-1} \left(1 - \frac{i}{s}\right) \geq \left(1 - \frac{r}{s}\right)^r \geq 1 - \frac{r^2}{s} \geq 1 - n^{-3/8}.$$

第二个不等号使用了 $(1 - x)^r \geq 1 - rx$, 其中 $0 \leq x \leq 1$ 、 r 为正整数, 最后一步代入了 $r < n^{1/4}$ 和 $s \geq n^{7/8}$ 。这个估计对满足第一条的每一份样本都成立, 因此也是第二件事的条件概率下界。

把两部分相乘, 得到算法的成功概率至少为 $(1 - e^{-n^{1/8}})(1 - n^{-3/8}) \geq 1 - e^{-n^{1/8}} - n^{-3/8}$ 。由于指数项 $e^{-n^{1/8}}$ 比 $n^{-3/8}$ 衰减得更快, 失败概率为 $O(n^{-3/8})$ 。□

14.6 MapReduce: 分布式数据处理

在本章的最后, 我们考虑一个更实际的并行计算问题, 假如要统计大量网页中各个单词的出现次数, 一台机器处理全部网页可能需要很长时间, 那么能否将网页分给多台机器, 各自处理一部分, 最后再合并结果呢? 解决思路并不复杂, 但实现时还要解决任务怎样分配、结果怎样传输, 以及某台机器发生故障后怎么办等问题。

Google 的 Jeffrey Dean 和 Sanjay Ghemawat 在 2004 年的 OSDI 论文中介绍了 MapReduce 编程模型及其实现。它将很多数据处理任务共同需要的分工方式提取出来, 让程序员主要编写 Map 和 Reduce 两个函数, 分别说明如何处理输入、如何合并结果。框架则负责将任务分配给不同机器, 传输中间结果, 并在工作节点故障后重新安排相关任务。这样, 设计新的数据处理程序时, 就可以集中考虑算法本身, 不必每次重新处理这些机器协作的问题。

14.6.1 从词频统计看计算过程

仍然以词频统计为例。假设输入文档已经分好词，Map 每读到一个词 w ，就输出 $(w, 1)$ ，表示这个词又出现了一次。这里的 w 称为键，1 称为值，所以输出的每一项都是一个键值对。由于处理某篇文档不需要等待其他文档，各个 Map 调用可以分给不同机器同时完成。

接下来要合并各台机器的结果。例如，机器甲读到每一次的 tree 和机器乙读到每一次的 tree，都应该算进 tree 的总次数中，因此框架将键相同的键值对汇集到一起，这个过程通常称为 shuffle。Reduce 得到一个词及其对应的全部计数后，只需将这些计数相加，就得到了该词的总次数。不同词的求和互不影响，所以这一步也可以分给多台机器完成。

算法 14.4 WordCount: 用 MapReduce 统计词频

输入： 文档集合，各文档已按相同规则分词

输出： 每个出现过的词及其总次数

```

1: procedure MAP(docid, words)
2:   for 文档中的每个词  $w$  do
3:     输出  $(w, 1)$ 
4:   end for
5: end procedure
6: 框架汇集所有 Map 输出，对每个词  $w$  调用一次 REDUCE( $w$ , counts)
7: procedure REDUCE( $w$ , counts)
8:    $s \leftarrow 0$ 
9:   for 属于词  $w$  的每项计数  $c$  do
10:     $s \leftarrow s + c$ 
11:   end for
12:   输出  $(w, s)$ 
13: end procedure

```

14.6.2 多台机器如何协作

刚才从算法上说明了 Map 和 Reduce 各做什么，下面再看看框架怎样安排它们。原论文的实现中，有一个主节点负责分配任务，其余机器作为工作节点执行任务。输入被划成若干份，主节点将尚未完成的任务分给空闲的工作节点。一台机器做完当前任务后可以继续领取下一项任务，所以任务数可以多于机器数。

每个 Map 任务先将输出保存在本地磁盘上，再按键分给不同的 Reduce 任务。例如，共有 R 个 Reduce 任务，编号为 $0, \dots, R-1$ ，就可以用散列函数 h ，将词 w 交给编号 $h(w) \bmod R$ 的任务。各台机器采用相同的散列函数，因此同一个词的计数会交给同一个任务。每个任务从各台机器取得属于自己的数据后，再按词排序，便能把同一个词的全部计数放到一起，逐词调用 Reduce。

如果某个工作节点失去响应，主节点会将需要重做的任务分配给其他节点。以词频统计为例，对同一份输入重新执行 Map 或 Reduce，结果不会改变，而框架只采用任务的一份有效输出，所以不会因为重试而重复计数。由此，部分工作节点发生故障时，也可以继续完成整项计算。

另外，网络传输往往是很大的开销。如果一份文档中 tree 出现了一万次，按前面的做法就要发

送一万个 $(tree, 1)$ 。因此可以先在本地相加，只发送 $(tree, 10000)$ ，最终统计结果也一样。这种局部合并称为 *combiner*，因为整数求和满足结合律，因此可以采用这一办法。即便如此，如果某个词对应的数据特别多，负责它的 *Reduce* 任务仍可能比其他任务慢，导致其他机器等待，这就是前面提到过的负载不均衡。

14.6.3 应用

词频统计只是一个简单例子。回忆倒排索引，每个词项需要保存出现过它的文档编号，因此可以让 *Map* 输出（词项，文档编号），框架按词项分组后，再由 *Reduce* 去重并整理成倒排表。类似地，要统计每个网页被访问了多少次，可以从访问日志中输出（网页地址，1），然后按地址求和。原论文也介绍了 Google 将 MapReduce 用于搜索索引构建的实践。

这些问题有一个共同特点，即每份输入都可以先独立处理，之后只需按键合并相关结果。因此，MapReduce 能将它们自然地分给多台机器完成。但如果一个算法需要反复交换大量中间数据，通信所花的时间也可能很长，因此使用 MapReduce 并不一定能很高效地加速计算。

14.7 习题

习题 14.1 (链表排名). 给定一条含 $n \geq 1$ 个结点的无环单链表，结点编号为 $1, \dots, n$ ，用数组 $next[v]$ 保存结点 v 的后继编号（编号不一定与链上的先后顺序相同），尾结点的后继为 NIL。我们希望求出每个结点到尾结点的距离，距离按经过的边数计算，这称为链表排名（list ranking）问题。例如，链表 $4 \rightarrow 1 \rightarrow 5 \rightarrow 2 \rightarrow 3$ 中，这五个结点到链尾的距离依次为 4, 3, 2, 1, 0。请设计一个 CREW 并行算法，在 $O(\log n)$ 深度内求出全部距离，并分析总工作量。要求算法保留原链表。

提示：如果让每个结点分别沿链表走到末尾，最长的一次需要走 $n - 1$ 步。为了加快这一过程，可以让各结点利用其他结点已经算出的距离，开始时只能向后跨过一条边，更新一轮后就能跨过两条，再更新一轮便能跨过四条，直到到达链尾。这种做法称为指针跳跃（pointer jumping）。

14.8 参考答案

习题 14.1. 用 $p[v]$ 记录从结点 v 出发，当前已经向后计算到的结点。例如， $p[v] = u$ 表示已经算出了从 v 到 u 的距离，这段距离保存在 $d[v]$ 中，按原链表中经过的边数计算。计算时只更新 p ，原链表的 $next$ 保持不变。开始时令 $p[v] = next[v]$ ，因此除尾结点外，每个结点的 $d[v]$ 都是 1。尾结点没有后继，令它的指针为 NIL、距离为 0。以后若某个结点的指针变为 NIL，就表示已经算完它到链尾的距离，将答案保存在 $d[v]$ 中。

每轮让所有结点同时更新。假设 v 当前指向 u ，而 u 指向更后面的一个结点，那么从 v 到那个结点的距离就是 $d[v] + d[u]$ 。因此，可以让 v 直接指向 $p[u]$ ，并把这两段距离相加： $p_{new}[v] = p[u]$ ， $d_{new}[v] = d[v] + d[u]$ 。因此每轮各结点同时做常数读写和加法，深度为 $O(1)$ 、工作量为 $O(n)$ 。合计 $O(\log n)$ 轮，得到深度 $O(\log n)$ 、总工作量 $O(n \log n)$ 。由于一个结点的距离可能同时被自己和前面的结点读取，这里需要使用 CREW 模型。□

14.9 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第十四讲。

2. Uzi Vishkin, *Thinking in Parallel: Some Basic Data-Parallel Algorithms and Techniques*. [作者讲义](#)第 2–4 章介绍并行计算模型、前缀和与归并, 第 8 章介绍寻找最大值的几种方法, 第 9.2 节介绍链表排名与指针跳跃。
3. Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein, *算法导论*, 第三版, 第 27 章介绍多线程算法及其工作量和深度分析。
4. Xian-He Sun、Lionel M. Ni, *Scalable Problems and Memory-Bounded Speedup*, *Journal of Parallel and Distributed Computing*, 19(1):27–37, 1993。第 4 节比较三种加速比模型, 并推导可用内存增加、问题随之扩大时的加速比公式。
5. Jeffrey Dean、Sanjay Ghemawat, *MapReduce: Simplified Data Processing on Large Clusters*, OSDI 2004。 [Google Research 原论文](#)第 2 节给出编程模型和词频例子, 第 3 节介绍任务如何分配给不同机器, 以及各台机器产生的同一个键的数据怎样交给 Reduce 处理。